Frida Sand
10. Exam Project 1
March 8, 2026

Reflection Report

Exam Project 1 - E-Commerce Application

Database ERD

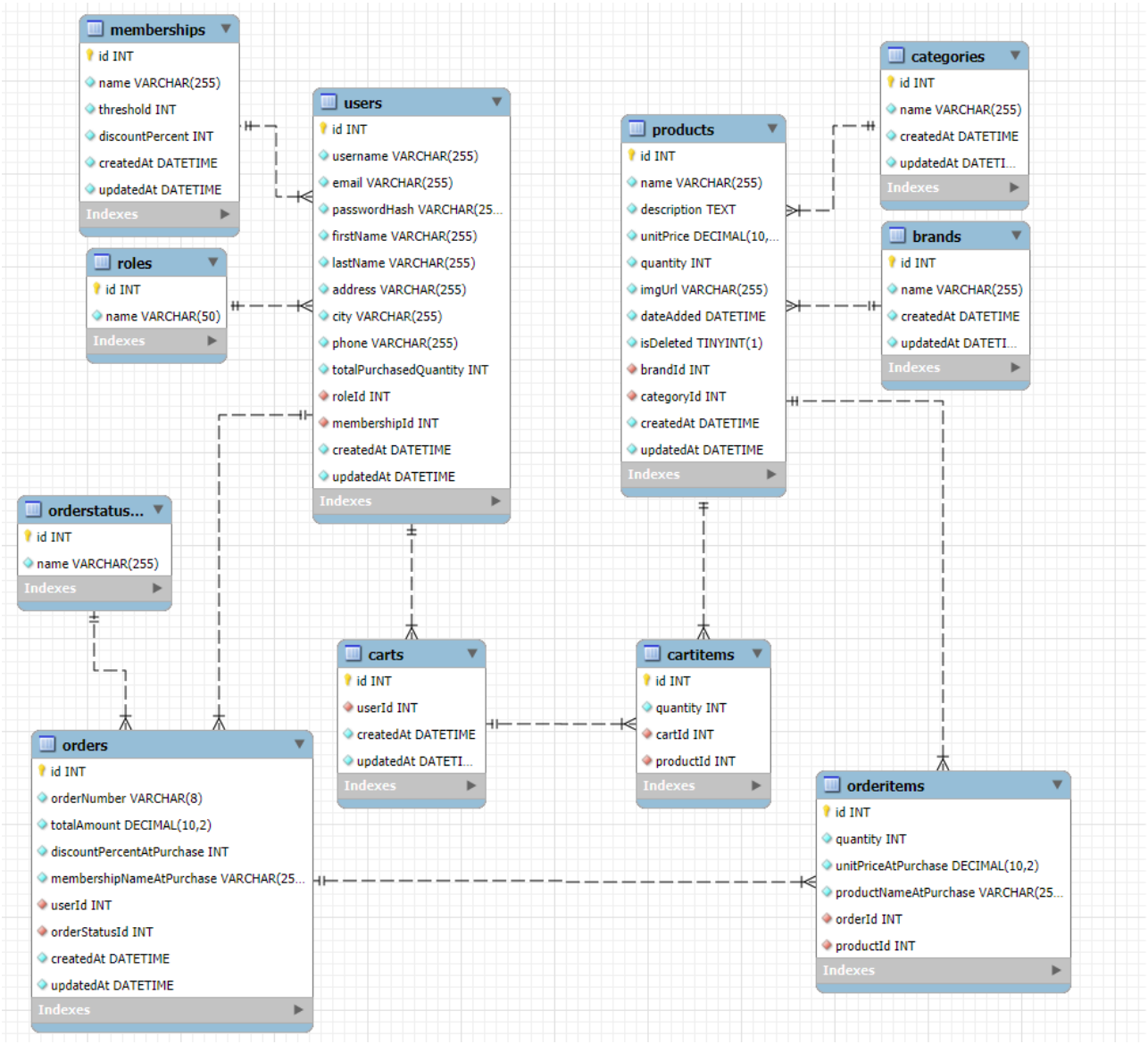

Figure 1. Complete database ERD showing tables, fields, and relationships.

Database Relationships

- A Role has many Users; therefore, this is a one-to-many relationship.
- A Membership has many Users; therefore, this is a one-to-many relationship.
- A User has many Orders; therefore, this is a one-to-many relationship.
- A User has one Cart; therefore, this is a one-to-one relationship.
- A Brand has many Products; therefore, this is a one-to-many relationship.
- A Category has many Products; therefore, this is a one-to-many relationship.
- A Cart has many CartItems; therefore, this is a one-to-many relationship.
- A Product has many CartItems; therefore, this is a one-to-many relationship.
- An OrderStatus has many Orders; therefore, this is a one-to-many relationship.
- An Order has many OrderItems; therefore, this is a one-to-many relationship.
- A Product has many OrderItems; therefore, this is a one-to-many relationship.

3NF Compliance

The database design follows Third Normal Form with some exceptions for audit requirements:

- **Historical snapshot fields:** OrderItem stores unitPriceAtPurchase and productNameAtPurchase, while Order stores discountPercentAtPurchase and membershipNameAtPurchase. These capture the values at purchase time, preserving what the customer actually paid rather than reflecting current prices or membership tiers.

- **Computed totalAmount field:** Order.totalAmount is calculated at checkout and stored, even though it could be derived from order items and discount. This violates 3NF but guarantees the exact amount charged is preserved without recalculation.

Project Progression

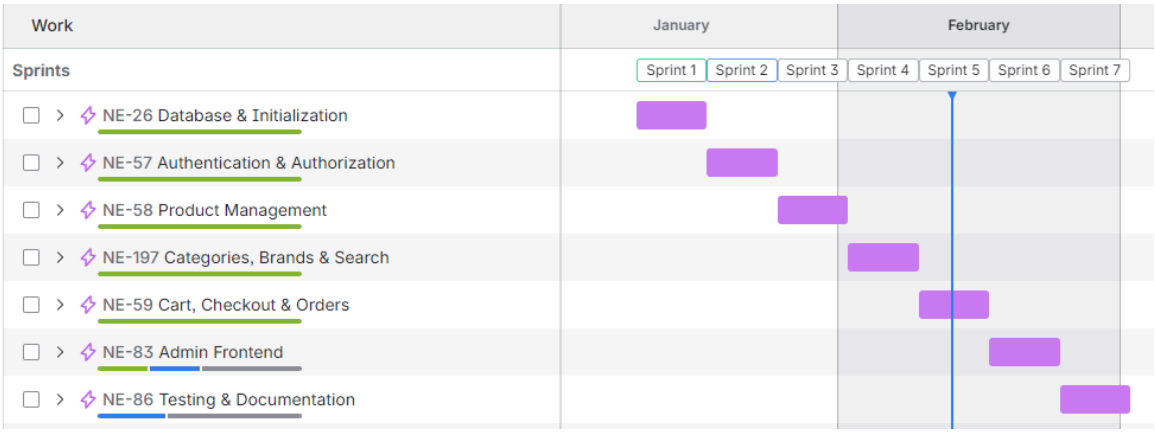

Figure 2: Jira Roadmap. Each epic contains stories and tasks broken into subtasks.

The project was organized in Jira with seven weekly sprints across seven epics, from database design through to testing and documentation.The plan was roughly followed, but sprint boundaries blurred as architectural decisions got revisited.

Database, initialization, and authentication

Database design came first as everything else depended on it. The Noroff API returns product data in a flat structure, so normalizing that into a 3NF design became the foundation that shaped the rest of the backend. My original design stored order status as a string directly on the order table. I later recognized that as a normalization gap, partly from feedback on a previous assignment(DAB), and replaced it with an OrderStatuses lookup table. Authentication and the /init seed endpoint followed, rounding out the early backend before CRUD work started.

Backend API and service layer

These sprints overlapped because each set of endpoints raised architectural questions I hadn't anticipated. I started with CRUD for simpler entities before moving to products, cart, checkout, and orders, building up thin controllers, orchestration services, Joi validation middleware, and centralized error handling along the way.

Admin frontend

I left the frontend until the backend was stable, then implemented session-based auth and a centralized apiClient helper for all fetch calls. The interface patterns went through a few iterations before I settled on inline editing for simpler entities and full-page forms for more complex ones.

Testing, documentation, and revisiting

The final sprint covered Jest/Supertest tests and Swagger documentation, while the last week was reserved for backlog work and reflection. During this period, I revisited the Order model and added totalAmount. Although the discount percentage was already stored on the order, the final discounted total was not. I decided to calculate and persist totalAmount during checkout as a deliberate denormalization. I had originally left it out to stay closer to 3NF, but later concluded that the final order total belonged on the historical purchase record.

Challenges

Checkout flow and service boundaries

The checkout process coordinated multiple concerns within one transaction, from stock validation and order snapshots through to membership recalculation. I considered orchestrating checkout directly in the controller, but that would have placed transaction management and cross-service coordination in the routing layer. Placing it in CartService would have mixed cart operations with ordering, stock, and membership concerns. I solved this by creating a dedicated CheckoutService to coordinate the flow in one place. This kept the transaction easier to reason about and gave the surrounding services clearer responsibilities. I used the same pattern for AuthService, which handles registration flow without making the user service responsible for membership assignment.

Cart creation and interpreting the brief

The brief states that registered users will have carts and orders, which I initially read as creating a cart at registration. After thinking through the flow more carefully, I realised that this would also create an unnecessary cart for the seeded admin account, even though the admin does not participate in the customer shopping flow. I chose lazy cart creation instead, using findOrCreate when the cart is first needed. This kept registration simpler, avoided unused rows, and still supported the required customer behaviour.

Optional authentication for role-based visibility

The brief requires guests to view products, but admins should also see soft-deleted ones. Standard auth middleware is binary: no token means rejected. Protecting the route for admin visibility would block guest access entirely. The solution was an optionalAuth middleware that parses a token if present but passes unauthenticated requests through with req.user set to null. The controller then checks the role to decide which products to return. I later caught that the search endpoint was missing the same middleware, so admin searches excluded soft-deleted products even though the main product listing showed them.

The general pattern behind most of my challenges was recognising when a solution was good enough and when further refinement was no longer useful. I know the rational approach is to meet the requirements, verify it works, and move on, but actually sticking to that while working is something I am still figuring out.
This project helped me practice drawing a better line between necessary refinement and overengineering.

ENV Examples

Back-end:

```
NODE_ENV=development
PORT=3000
DIALECT=mysql
HOST=localhost
DATABASE_NAME=noroff_ep_ecommerce
ADMIN_USERNAME=admin
ADMIN_PASSWORD=admin123
JWT_SECRET=thisisverysecret
JWT_EXPIRES_IN=2h
NOROFF_PRODUCTS_URL=http://backend.restapi.co.za/items/products
```

Front-end:

```
NODE_ENV=development
PORT=3001
SESSION_SECRET=verysecretsecret
API_URL=http://localhost:3000
```